

PymeCore

Mini ERP para gestión comercial, stock y facturación

Proyecto personal en ASP.NET Core MVC para portfolio .NET Junior

Resumen del proyecto

PymeCore es una aplicación web de gestión empresarial para pequeñas empresas. El objetivo es controlar clientes, proveedores, productos, stock, presupuestos, pedidos y facturas desde una interfaz web desarrollada con ASP.NET Core MVC. El proyecto busca demostrar dominio práctico de .NET MVC, Entity Framework Core, base de datos relacional, validaciones, ViewModels, servicios y lógica de negocio real.

Objetivo principal

Construir un proyecto .NET más completo que un CRUD básico, con un flujo empresarial reconocible y defendible en entrevistas. La aplicación debe parecer una herramienta interna que una PYME podría usar para controlar ventas, productos y facturación.

- Reforzar el perfil principal: C#, ASP.NET Core MVC y Entity Framework Core.
- Crear una aplicación visual y navegable con Razor Views y Bootstrap.
- Demostrar relaciones entre entidades, reglas de negocio y control de estados.
- Preparar un repositorio de GitHub profesional con README, capturas y explicación técnica.

Tipo de proyecto

No se plantea como un ERP completo tipo Odoo, SAP o Holded. La idea correcta para un portfolio junior es un mini ERP: más completo que un CRM, pero con alcance controlado para poder terminarlo y defenderlo bien.

No es	Sí es
Un ERP real con contabilidad completa, pagos, múltiples almacenes e integración fiscal.	Un mini ERP centrado en gestión comercial, stock y facturación básica.
Un proyecto infinito con demasiadas tecnologías.	Un proyecto .NET MVC serio, cerrado, visual y explicable.
Otro CRUD simple.	Una aplicación con flujo de negocio y relaciones reales.

1. Idea general del negocio

La aplicación representa el funcionamiento básico de una pequeña empresa que vende productos o servicios. La empresa compra o registra productos, controla el stock disponible, crea presupuestos para clientes, convierte presupuestos aceptados en pedidos y finalmente genera facturas.

Proveedor -> Producto -> Stock

Cliente -> Presupuesto -> Pedido -> Factura

Explicación sencilla

Por un lado, la empresa controla lo que tiene: proveedores, productos y stock. Por otro lado, controla lo que vende: clientes, presupuestos, pedidos y facturas.

Ejemplo de uso real

1. Se registra un proveedor: TechDistribuciones S.L.
2. Se registra un producto: Monitor Dell 24 pulgadas, con precio de coste, precio de venta, stock actual y stock mínimo.
3. Se registra un cliente: Clínica Dental López.
4. El cliente pide presupuesto para comprar dos monitores y un servicio de instalación.
5. Se crea un presupuesto con varias líneas y el sistema calcula el total.
6. El cliente acepta el presupuesto y se genera un pedido de venta.
7. Al completar el pedido, el sistema descuenta el stock correspondiente.
8. Finalmente se genera una factura con base imponible, IVA y total.

Qué problema resuelve

- Evita tener clientes, productos, stock y facturas repartidos en hojas de cálculo.
- Permite saber qué productos están bajos de stock.
- Permite seguir el estado de presupuestos, pedidos y facturas.
- Centraliza la información comercial básica de una PYME.
- Muestra un flujo de trabajo empresarial real, no pantallas aisladas.

Pantalla principal: Dashboard

La aplicación tendrá un panel inicial con métricas de negocio. Esta pantalla es importante porque hace que el proyecto parezca más profesional y ayuda a explicar la aplicación rápidamente.

- Clientes activos.
- Productos registrados.
- Productos con bajo stock.
- Presupuestos pendientes o enviados.
- Facturas pendientes de pago.
- Ventas estimadas o facturadas del mes.

2. Módulos de gestión interna

Estos módulos representan la parte interna de la empresa: proveedores, productos y control de stock. Son importantes porque convierten el proyecto en algo más parecido a un ERP y no solo en una aplicación de clientes.

Clientes

Permite guardar empresas o personas a las que se vende. Cada cliente tendrá datos básicos y un estado activo/inactivo para evitar borrados físicos innecesarios.

Campo	Descripción
Nombre	Nombre comercial o razón social del cliente.
NIF/CIF	Identificador fiscal. Puede ser opcional en el MVP.
Email y teléfono	Datos principales de contacto.
Dirección y ciudad	Información útil para facturación y gestión.
Estado	Activo o inactivo.

Proveedores

Permite guardar empresas a las que se compran productos o materiales. Esto ayuda a relacionar productos con proveedores y da más sentido empresarial al proyecto.

- Crear, editar y consultar proveedores.
- Buscar por nombre o CIF.
- Asociar productos a un proveedor.
- Mantener persona de contacto, email y teléfono.

Productos

El catálogo de productos o servicios es uno de los módulos centrales del mini ERP. Cada producto tendrá información comercial y de stock.

Dato del producto	Ejemplo
Nombre	Monitor Dell 24 pulgadas
Referencia/SKU	MON-DELL-24
Precio de coste	110 EUR
Precio de venta	159 EUR
Stock actual	8 unidades
Stock mínimo	3 unidades
Proveedor	TechDistribuciones S.L.

Movimientos de stock

Para que el proyecto sea más profesional, el stock no debería modificarse simplemente cambiando un número. La aplicación registrará movimientos de stock.

Tipo de movimiento	Efecto
Entrada	Aumenta el stock, por ejemplo al recibir productos.
Salida	Reduce el stock, por ejemplo por venta.
Ajuste	Corrige diferencias manuales.
Devolución	Aumenta el stock por devolución de producto.

Regla defendible en entrevista

El stock actual se actualiza mediante movimientos. Esto permite tener histórico y explicar por qué un producto tiene una cantidad determinada.

3. Módulos de venta y facturación

Estos módulos representan el flujo comercial principal: el cliente pide algo, la empresa hace un presupuesto, el presupuesto se convierte en pedido y finalmente se genera una factura.

Cliente -> Presupuesto -> Pedido -> Factura

Presupuestos

Un presupuesto es una propuesta de venta antes de que el cliente acepte. Tendrá una cabecera y varias líneas de detalle.

- Datos principales: número, cliente, fecha, estado y observaciones.
- Estados: borrador, enviado, aceptado y rechazado.
- Líneas: producto o servicio, cantidad, precio unitario y subtotal.
- Total calculado automáticamente a partir de las líneas.

Regla de negocio	Motivo
No aceptar un presupuesto sin líneas.	Evita presupuestos vacíos.
El total no se escribe manualmente.	Se calcula desde cantidades y precios.
Un presupuesto rechazado no genera pedido.	Respetar el flujo comercial.

Pedidos de venta

Cuando un presupuesto es aceptado, se puede convertir en pedido. El pedido representa una venta comprometida que puede estar pendiente, en preparación, completada o cancelada.

- El pedido conserva el cliente y las líneas del presupuesto.
- Permite controlar el estado de preparación de la venta.
- Al completar el pedido, se registran salidas de stock.
- Un pedido cancelado no debería generar factura.

Facturas

La factura se genera a partir de un pedido completado. No se plantea facturación legal avanzada, sino una facturación básica suficiente para portfolio.

Dato	Ejemplo
Número de factura	FAC-2026-001
Cliente	Clínica Dental López
Base imponible	468,00 EUR
IVA	21%
Total	566,28 EUR
Estado	Pendiente, pagada o anulada

Lógica importante

No se debe poder generar dos veces la misma factura para el mismo pedido. El total y el IVA se calculan automáticamente.

4. Enfoque técnico en ASP.NET Core MVC

El proyecto debe reforzar el perfil .NET. Por eso la tecnología principal será ASP.NET Core MVC con Razor Views, no React ni Blazor. React ya queda cubierto con otro proyecto del portfolio.

Área	Tecnología / enfoque
Backend principal	C#, ASP.NET Core MVC
Interfaz	Razor Views + Bootstrap
Base de datos	PostgreSQL o SQL Server
ORM	Entity Framework Core
Validaciones	DataAnnotations + ViewModels
Arquitectura	Controladores, servicios, modelos y ViewModels
Autenticación	ASP.NET Core Identity con roles en fase avanzada
Repositorio	GitHub con README profesional, capturas y guía de ejecución

Estructura conceptual

- Models: entidades de base de datos como Cliente, Proveedor, Producto, Presupuesto, Pedido y Factura.
- ViewModels: objetos específicos para formularios y pantallas, evitando usar siempre la entidad directamente.
- Controllers: reciben peticiones, coordinan acciones y devuelven vistas.
- Services: contienen la lógica de negocio importante, por ejemplo crear pedido desde presupuesto o actualizar stock.
- Views: pantallas Razor para listar, crear, editar, ver detalles y confirmar acciones.

Fases recomendadas

Fase	Contenido
1. Base del proyecto	Crear solución MVC, layout, Bootstrap, conexión a base de datos y estructura inicial.
2. Gestión interna	Clientes, proveedores, productos y movimientos de stock.
3. Ventas	Presupuestos con líneas, estados y cálculo automático de totales.
4. Pedidos y facturas	Conversión de presupuesto aceptado a pedido, descuento de stock y generación de factura.
5. Profesionalización	Dashboard, filtros, paginación, roles, README, capturas y despliegue.

Alcance que NO entra al principio

- Contabilidad real.
- Integración con Hacienda o facturación electrónica oficial.
- Pagos online.
- PDFs perfectos desde el primer día.
- Multiempresa avanzada.
- Múltiples almacenes complejos.
- Microservicios, CQRS o arquitectura demasiado empresarial.

Valor para el portfolio

PymeCore permite explicar que se ha construido una aplicación empresarial en .NET MVC con entidades relacionadas, formularios, validaciones, control de estados, lógica de negocio, base de datos relacional y una interfaz visual. Es un proyecto más sólido que un CRUD simple y complementa bien otros proyectos de React, TypeScript y APIs REST.

Frase para CV o LinkedIn

PymeCore - Mini ERP desarrollado con ASP.NET Core MVC, Entity Framework Core y base de datos relacional para gestionar clientes, proveedores, productos, stock, presupuestos, pedidos y facturas, incluyendo ViewModels, validaciones, servicios, lógica de negocio y dashboard.